CYBR 4550 · OPERATION CANDLELIGHT · TRACK A (RED TEAM)

Birthday Memory Application Security Assessment

Board report: findings, evidence and remediation plan

Prepared by	Michael Tesfaye
Course	CYBR 4550, Utah Valley University
Test date	November 7, 2026
Target	Birthday Memory (React, Express, PostgreSQL in Docker), local deployment
Repository	github.com/Zmcmike/cyber-uvu-student-projects, branch `security-assessment-redteam`, folder `cybr-4550/threat_actors/birthday_memory/security`
Methodology	OWASP Web Security Testing Guide, mapped to OWASP Top 10 2021; CVSS v3.1
Data	Synthetic only. No real personal data was used or exposed.

1. Executive summary

Anyone who can reach this application over the network can read, change and delete every record, and the database behind it accepts a published default superuser login. Neither takes any skill: both were reproduced with single commands.

I tested the application on one machine, using its own Docker daemon, with invented data. I ran 30 numbered tests across the OWASP Top 10, the database, the container and the dependencies, which produced 39 timestamped screenshots (plus 3 baseline captures). **Nine findings are confirmed: two Critical, one High, four Medium and two Low.** Four categories were tested and held up, and I say so plainly in section 5, because a security review that only lists failures overstates the risk.

ID	Finding	Severity	CVSS
F-01	No authentication or authorization on any API route	**Critical**	9.8
F-03	Database on all interfaces with default superuser credentials	**Critical**	9.8
F-08	Vulnerable dependencies (5 advisories)	**High**	7.5
F-05	No transport encryption; certificate checks disabled when TLS is on	**Medium**	6.8
F-04	Personal data stored in plaintext, readable from the Docker volume	**Medium**	5.5
F-02	No audit logging: a deletion leaves no trace	**Medium**	5.3
F-07	No rate limiting and no pagination	**Medium**	5.3
F-09	Container not hardened, image not pinned, 62 image CVEs	**Low**	3.9
F-06	Wildcard CORS, no security headers, framework disclosed	**Low**	3.1

What to do first

1. **Bind the database to loopback and replace its credentials** (`"127.0.0.1:5544:5432"`, generated secret, least-privilege role). One line of config removes the largest exposure.
2. **Add authentication and per-record authorization** to every API route, and stop returning phone and email to callers who do not need them.
3. **Add audit logging** for every write, with who, what and when.
4. **Turn on TLS** end to end and set certificate verification to on.
5. **Update dependencies and pin the database image by digest.**

> **How certain are I?** Every finding is backed by a timestamped request, command output or scanner report. Where reachability depends on something I did not test (a second device on the network, exploitation of a dependency), the finding says so. Scanner severities are shown beside my own assessment where they differ.

2. Scope, rules and method

Scope and rules of engagement

- Target: the Birthday Memory application running on my own computer (API on port 4000, web app on 5173, PostgreSQL 16 in Docker on 5544).
- Localhost and my own Docker daemon only. No other device, no campus network, no tunnelling. Testing from a second device on the LAN was out of scope.
- Synthetic data only. Test records created during testing were deleted afterwards (the database was returned to its 8 seeded rows).
- No denial-of-service testing beyond a 30-request burst. No exploitation of dependency vulnerabilities. No attempt to escape the container.

Method

I followed the OWASP Web Security Testing Guide and tracked coverage against all ten OWASP Top 10 2021 categories, plus container-specific and availability checks. Each category ended with a verdict: Confirmed, Not Vulnerable (tested), Not Applicable, or Not Tested with a reason. The full matrix is in the repository (`security/EVIDENCE-LOG.md`).

Evidence standard

A finding is reported only when a request and its response, a command output or a scanner report proves it. Every capture shows a timestamp and the command that produced it, and has a matching raw output file. A hypothesis that did not prove out is listed in section 5 as not confirmed, not reported as a finding.

Scoring

CVSS v3.1 base scores, calculated with a script from the vector (not by hand), with a one-sentence justification for each metric. CVSS suits missing controls poorly (for example, absent logging), so those are scored as enabling weaknesses and labelled that way.

Tools

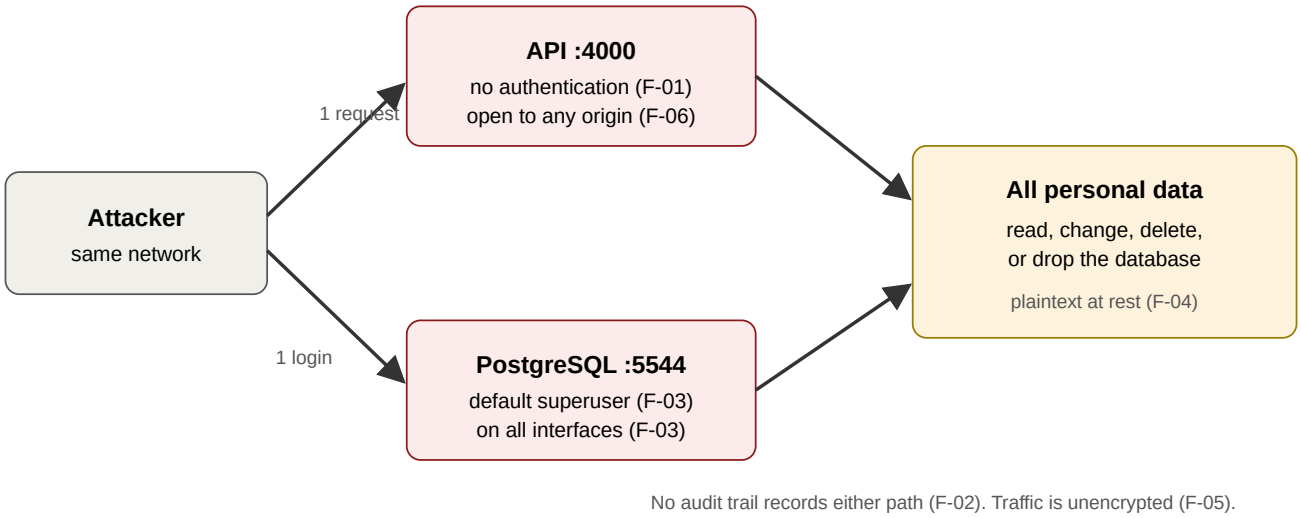
PowerShell and `curl.exe` for API requests; `psql` inside the container for database tests; `docker inspect`, `docker port` and a read-only throwaway container for container and volume checks; `npm audit`, gitleaks and Trivy (both run in Docker) for scanners; PowerShell `Test-NetConnection` for the port check (nmap was not installed). AI assistance is disclosed in Appendix C.

Timestamps

Screenshots show local time (Mountain Standard Time). Container and database logs are in UTC, 7 hours ahead.

3. System overview and attack chain

Birthday Memory stores a person's first and last name, birthdate, phone and email. A React web app (port 5173, listening on IPv6 loopback only) calls an Express API (port 4000) which queries PostgreSQL 16 running in Docker (port 5544 on the host). The API and the database both listen on all network interfaces. The API has no login, and the database ships with `birthday/birthday` as its superuser.



Two independent routes lead to the same data, which is why the first fix on the list matters but is not enough alone: closing the database port still leaves the API open, and adding API authentication still leaves the database reachable.

> **Caveat (Certain):** I proved both services accept connections on my LAN address (192.168.0.47). I did not connect from a second device, so whether a given network would let another machine reach them depends on its firewall.

4. Findings

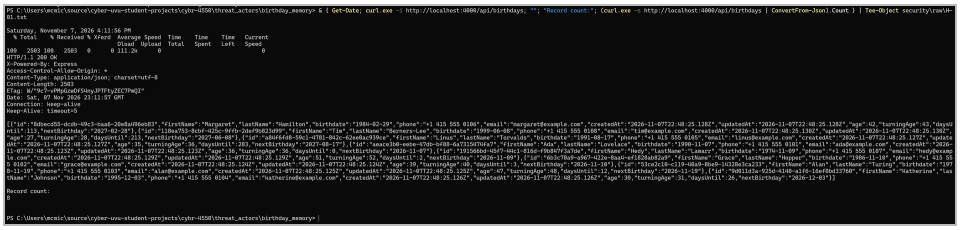
Each finding lists its evidence IDs. Files named by ID are in `security/screenshots` and `security/raw`. Confidence is stated as Certain (proven by the evidence shown), Likely (supported but not fully proven) or Guessing.

F-01 No authentication or authorization on any API route

Critical 9.8

Category: A01 Broken Access Control, A07 Authentication Failures, A04 Insecure Design (root cause)
Vector: `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`
Component: server/src/index.js, server/src/routes.js

What I found. Every route answers without credentials. I read all 8 records (H-01), created a record (HTTP 201), updated it (200) and deleted it (204) (H-02a, H-02b, H-02c), all with no login. The code has no login route and no auth middleware (H-15), and no roles, record ownership or data minimization anywhere (H-24). Confidence: Certain.



H-01: GET /api/birthdays with no credentials returns all records



H-02c: DELETE with no credentials returns 204 (create and update: H-02a, H-02b)

```
index.js:10: app.use(cors());
index.js:11: app.use(express.json());
index.js:22: app.use('/api/birthdays', birthdaysRouter);
```

Metric	Value	Why
AV	N	The API listens on all interfaces and answers plain HTTP.
AC	L	One curl call, no conditions.
PR	N	No account, token or login exists, so PR:L (a low-privilege account) does not apply.
UI	N	No victim action needed.
S	U	Impact stays in the app's own data.
C	H	Every name, birthdate, phone and email is returned.
I	H	Any caller can create, change or delete records.
A	H	Any caller can delete any record, using ids from the list call.

Impact. Complete loss of confidentiality, integrity and availability of the data to anyone who can reach port 4000.

Fix. Require authentication on every route; add a per-record owner and role checks; return only the columns each caller needs (`SELECT_COLUMNS` in `routes.js` line 10 returns phone and email to everyone).

F-03 Database on all interfaces with default superuser credentials

Critical 9.8

Category: A05 Security Misconfiguration, A01
Vector: `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`
Component: docker-compose.yml lines 8, 9, 11; server/src/db.js lines 11, 12

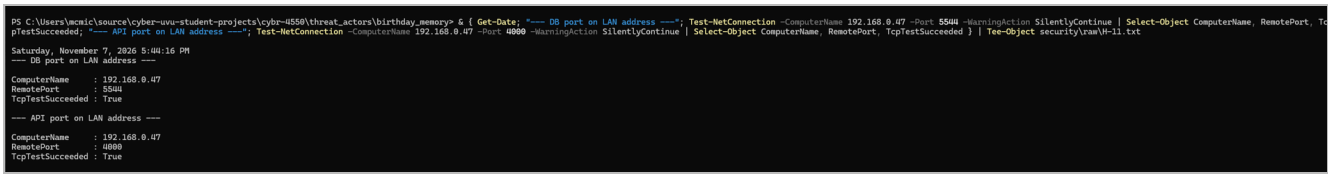
What I found. Logging in with `birthday/birthday` works (H-08a), and the same values are hard-coded in `db.js` and `docker-compose.yml` (H-08b, H-14). The role is a superuser (H-09). Port 5544 is published on `0.0.0.0` (H-12) and answered on the machine's LAN address (H-11). The password can also be read from the running container with `docker inspect` (H-19). Confidence: Certain for each fact; Likely that another device could connect, since I did not test from a second device.



H-09: the application role is a superuser (rolsuper = t)



H-12: docker port shows the database on all interfaces



H-11: ports 5544 and 4000 accept connections on the LAN address

```
db.js:11: PGUSER = 'birthday',
db.js:12: PGPASSWORD = 'birthday',
docker-compose.yml:8: POSTGRES_USER: birthday
docker-compose.yml:9: POSTGRES_PASSWORD: birthday
```

Metric	Value	Why
AV	N	Published on 0.0.0.0 and reachable on the LAN address.
AC	L	The credentials are published defaults in the repository.
PR	N	No account is needed to know a public default password.
UI	N	No victim action.
S	U	No escape from the container was demonstrated.
C	H	A superuser reads every table.
I	H	A superuser changes any data.
A	H	A superuser can drop the database.

Impact. Anyone on the same network could log in as a superuser and bypass the API entirely.

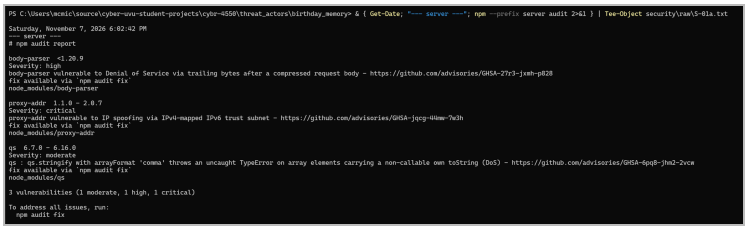
Fix. Publish as `127.0.0.1:5544:5432`; use a generated secret from a secrets store; create a least-privilege application role (no superuser, no createdb, no createrole); remove fallback credentials from `db.js`.

F-08 Vulnerable dependencies

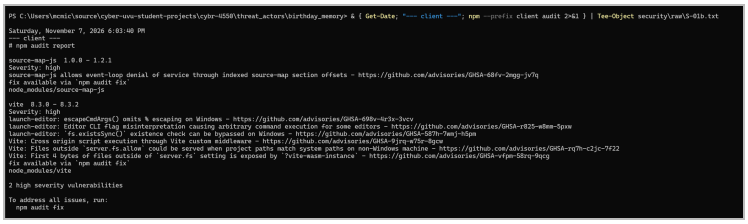
High 7.5

Category: A06 Vulnerable and Outdated Components
Vector: `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`
Component: server/package.json (Express 4.21.2 and dependencies), client/package.json (Vite)

What I found. `npm audit` reports 3 vulnerabilities on the server (1 critical, 1 high, 1 moderate) and 2 high on the client (S-01a, S-01b). A fix is available for all five. None was exploited. Confidence: Certain for the scanner output; Likely for the reachability column below.



S-01a: npm audit, server



S-01b: npm audit, client

Package	Scanner	My reachability assessment
body-parser < 1.20.9	High	Likely reachable: `express.json()` parses every request. Denial of service. Scored here.
proxy-addr 1.1.0 to 2.0.7	Critical	Not reachable as configured: the app does not set `trust proxy`. The critical label overstates the risk here.
qs 6.7.0 to 6.16.0	Moderate	Affects `qs.stringify` with one option; the app code does not use it.
vite 8.3.0 to 8.3.2	High	Development server only, and it listens on IPv6 loopback only (S-04b), so it is not exposed to the network.
source-map-js 1.0.0 to 1.2.1	High	Build and development tooling only.

Metric	Value	Why
AV	N	Triggered by a crafted request.
AC	L	A single crafted compressed body.
PR	N	The API has no authentication (F-01).
UI	N	No victim action.
S	U	Stays within the app.
C	N	Denial of service only.
I	N	Denial of service only.
A	H	A crash or stall takes the API offline.

Fix. Run `npm audit fix`, commit lockfiles, and add `npm audit` to CI.

F-05 No transport encryption; certificate checks disabled when TLS is on

Medium 6.8

Category: A02 Cryptographic Failures
Vector: `CVSS:3.1/AV:A/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N`
Component: server/src/db.js lines 14, 17, 20, 27; server/src/index.js

What I found. PostgreSQL reports `ssl = off` and the test session shows `ssl = f` (H-17). The API serves plain `http://`. In `db.js`, TLS only turns on when `PGSSL === 'true'`, and then with `rejectUnauthorized: false`, so the database certificate is not verified (H-18). Confidence: Certain for configuration and code; no traffic was intercepted.



H-17: SHOW ssl returns off; the API logs a plain http address



H-18: db.js line 17 sets rejectUnauthorized: false

Metric	Value	Why
AV	A	The attacker must share the network segment.
AC	H	The attacker must be positioned between client and server.
PR	N	No privileges on the app are needed.
UI	N	No victim action.
S	U	Stays within the app and its database.
C	H	Contact data and credentials cross the network in clear text.
I	H	Unprotected traffic can be altered in transit.
A	N	No availability effect.

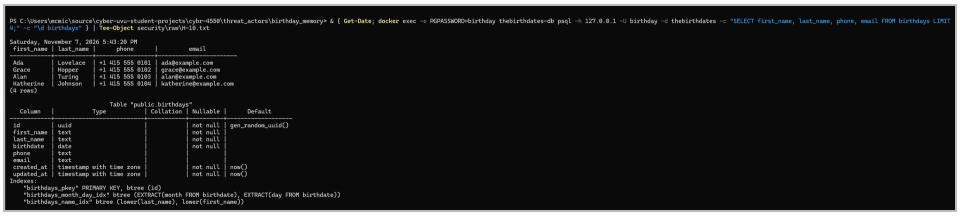
Fix. Terminate TLS in front of the API, require TLS for Postgres, and set `rejectUnauthorized: true` with a trusted certificate authority.

F-04 Personal data stored in plaintext, readable from the Docker volume

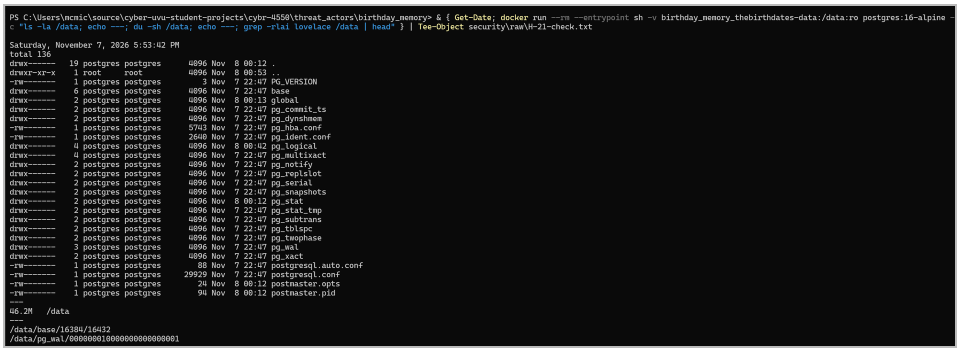
Medium 5.5

Category: A02 Cryptographic Failures
Vector: `CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N`
Component: birthdays table; Docker volume birthday_memory_thebirthdates-data

What I found. Names, phones and emails sit in plain `text` columns (H-10). Mounting the volume read-only in a throwaway container and searching its files found a seeded surname in the table file and in the write-ahead log (H-21). My first search, for an exact email, returned nothing and I do not know why; the broader case-insensitive name search found the data. Confidence: Certain.



H-10: plaintext columns and rows



H-21: surname found in the table file and in the write-ahead log

```
/data/base/16384/16432
/data/pg_wal/000000010000000000000001
```

Metric	Value	Why
AV	L	Needs access to the host to mount the volume.
AC	L	A simple grep.
PR	L	Needs the ability to run Docker or read the volume.
UI	N	No user action.
S	U	Stays on the host.
C	H	All personal data is readable.
I	N	Read only.
A	N	Read only.

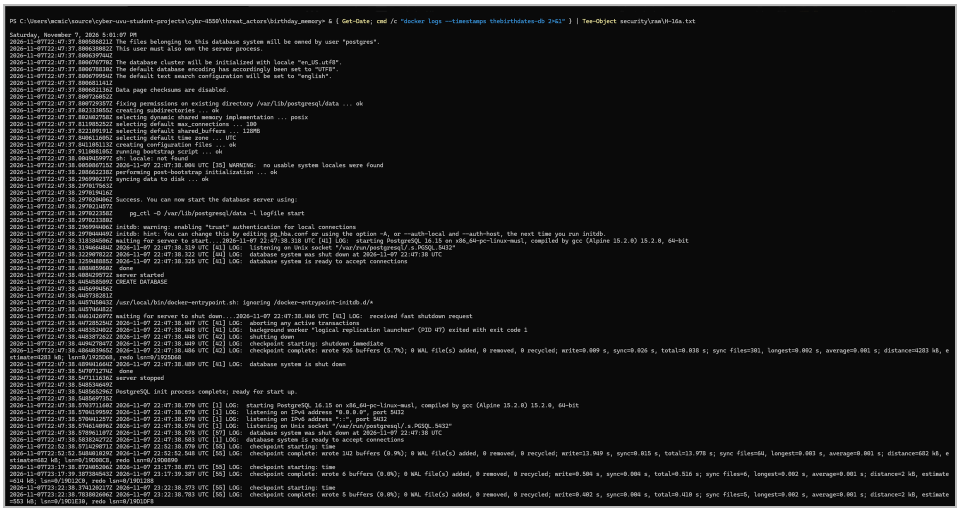
Fix. Encrypt the volume or disk, encrypt backups, consider column-level encryption for sensitive fields, and restrict who can run Docker on the host.

F-02 No audit logging: a deletion leaves no trace

Medium 5.3

Category: A09 Logging and Monitoring Failures
Vector: `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:N`
Component: server/src/*.js, Postgres container logs

What I found. After a create, an update and a delete, the database log held only startup lines and periodic checkpoint messages: no who, what or when (H-16a). The server code contains 9 `console` calls, all startup, error or seed messages, and nothing that records a request (H-16b). Confidence: Certain.



H-16a: database log after the delete (UTC timestamps, 7 hours ahead of local time)



H-16b: the only logging in the server code

Metric	Value	Why
AV	N	Applies to every network request.
AC	L	Nothing is needed to take advantage.
PR	N	Applies to unauthenticated callers.
UI	N	No user action.
S	U	Stays within the app.
C	N	Logging gaps do not expose data by themselves.
I	L	Changes cannot be attributed, which weakens the integrity of the record of events.
A	N	No availability effect.

Note. CVSS fits a missing control poorly. This is scored as an enabling weakness: it hides the effects of F-01 and F-03.

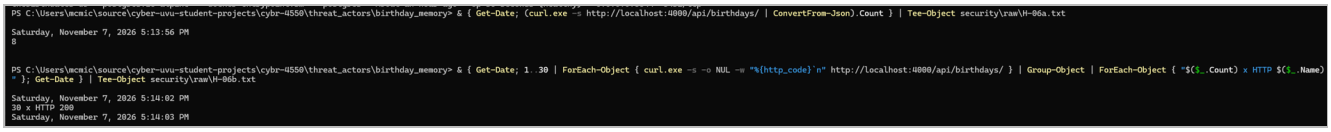
Fix. Log every request with a request id, caller and action; log all writes; ship logs to a store the app cannot alter; enable Postgres statement logging for writes.

F-07 No rate limiting and no pagination

Medium 5.3

Category: Availability, A04 Insecure Design
Vector: `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L`
Component: server/src/routes.js (GET /)

What I found. Thirty rapid requests all returned HTTP 200 with no throttling, and the list call returns every record with no limit parameter (H-06). I kept the burst small on purpose and did not test impact at scale. Confidence: Certain for the missing controls; Guessing for real-world impact at scale.



H-06: record count and a 30-request burst, all HTTP 200

Metric	Value	Why
AV	N	Network requests.
AC	L	Simple repeated requests.
PR	N	No account needed.
UI	N	No user action.
S	U	Stays within the app.
C	N	No confidentiality effect.
I	N	No integrity effect.
A	L	Resource use grows with table size and request volume; only a potential reduction was shown, not an outage.

Fix. Add rate limiting, pagination with a maximum page size, and a request body size limit.

F-09 Container not hardened, image not pinned, no SBOM, 62 image CVEs

Low 3.9

Category: A05, A08 Software and Data Integrity Failures, A06
Vector: `CVSS:3.1/AV:L/AC:H/PR:H/UI:N/S:U/C:L/I:L/A:L`
Component: docker-compose.yml line 3; the running container

What I found. The container runs with no capabilities dropped, a writable root filesystem and no memory, CPU or process limits (H-13); it is not privileged, which is the one good result. The image is referenced by a mutable tag, `postgres:16-alpine`, and no SBOM exists in the repository (H-23). Trivy reports 62 CVEs in the image: 1 in operating system packages and 61 in the bundled `gosu` binary, including 1 critical and 25 high (S-03). Confidence: Certain.

```
PS C:\Users\mcmic\source\cyber-uvu-student-projects\cybr-4550\threat_actors\birthday_memory> & { Get-Date; docker inspect thebirthdates-db --format "User={{.Config.User}} Privileged={{.HostConfig.Privileged
}} CapDrop={{.HostConfig.CapDrop}} ReadonlyRootfs={{.HostConfig.ReadonlyRootfs}} Memory={{.HostConfig.Memory}} NanoCpus={{.HostConfig.NanoCpus}} PidsLimit={{.HostConfig.PidsLimit}} SecurityOpt={{.HostConfig
.SecurityOpt}}" } | Tee-Object security\raw\H-13.txt

Saturday, November 7, 2026 5:58:47 PM
User= Privileged=false CapDrop=<no value> ReadonlyRootfs=false Memory=0 NanoCpus=0 PidsLimit=<no value> SecurityOpt=<no value>
```

H-13: docker inspect shows no hardening settings

```
PS C:\Users\mcmic\source\cyber-uvu-student-projects\cybr-4550\threat_actors\birthday_memory> & { Get-Date; "--- image tag in compose ---"; Select-String -Path docker-compose.yml -Pattern "image:" | ForEach-
Object { "docker-compose.yml:$($_.LineNumber): $($_.Line.Trim())" }; "--- pinned digest of the running image ---"; docker inspect thebirthdates-db --format "{{.Image}}"; docker image inspect postgres:16-alp
ine --format "{{index .RepoDigests 0}}"; "--- SBOM / lock files in repo ---"; Get-ChildItem -Recurse -Include "*sbom*","*.spdx*","*cyclonedx*","bom.json" -ErrorAction SilentlyContinue | Where-Object { $_.Fu
llName -notmatch "node_modules" } | Select-Object -ExpandProperty Name; "(end of SBOM search)" } | Tee-Object security\raw\H-23.txt

Saturday, November 7, 2026 6:02:02 PM
--- image tag in compose ---
docker-compose.yml:3: image: postgres:16-alpine
--- pinned digest of the running image ---
sha256:4278d7b1e3deb3193ddc803158f6532963f059fc25a2eefaee4b5a9e71741873
postgres@sha256:aa6cb33cf4c19c765cefcf48d96176258800870a9a1e930ad126200f9d825139
--- SBOM / lock files in repo ---
(end of SBOM search)
```

H-23: mutable image tag, digest not pinned, no SBOM file

```
PS C:\Users\mcmic\source\cyber-uvu-student-projects\cybr-4550\threat_actors\birthday_memory> & { Get-Date; docker run --rm aquasec/trivy:latest image --scanners vuln postgres:16-alpine 2>&1 | Out-File -Enco
ding utf8 security\raw\S-03-full.txt; Select-String -Path security\raw\S-03-full.txt -Pattern "Report Summary|Total:|postgres:16-alpine|gobinary|alpine " | ForEach-Object { $_.Line.Trim() } } | Tee-Object s
ecurity\raw\S-03.txt

Saturday, November 7, 2026 6:06:20 PM
2026-11-08T01:06:46Z    INFO    [gobinary] Detecting vulnerabilities...
Report Summary
┌─────────────────────────────────────┬────────┬─────────┬─────────┐
│               Target                │  Type  │ Vulns   │ Secrets │
├─────────────────────────────────────┼────────┼─────────┼─────────┤
│ postgres:16-alpine (alpine 3.24.4)  │ alpine │    1    │    -    │
├─────────────────────────────────────┼────────┼─────────┼─────────┤
│ usr/local/bin/gosu                  │ gobinary │   61    │    -    │
└─────────────────────────────────────┴────────┴─────────┴─────────┘
postgres:16-alpine (alpine 3.24.4)
Total: 1 (UNKNOWN: 0, LOW: 0, MEDIUM: 1, HIGH: 0, CRITICAL: 0)
usr/local/bin/gosu (gobinary)
Total: 61 (UNKNOWN: 1, LOW: 3, MEDIUM: 31, HIGH: 25, CRITICAL: 1)
```

S-03: Trivy totals by component

Metric	Value	Why
AV	L	Needs local or host-level access.
AC	H	Several preconditions, and gosu is not exposed to network input.
PR	H	Needs privileged access on the host.
UI	N	No user action.
S	U	No scope change shown.
C	L	Limited effect.
I	L	Limited effect.
A	L	Limited effect.

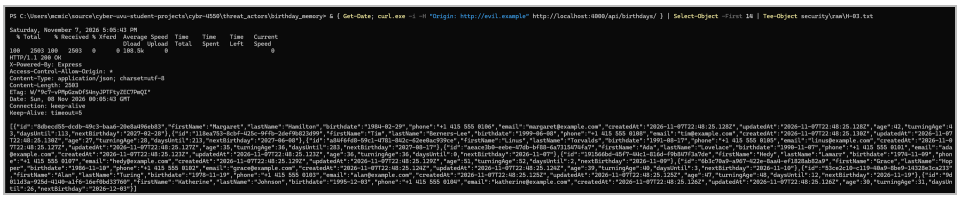
Reachability matters. The raw CVE count looks alarming, but gosu runs once at startup to drop privileges. I report both numbers. **Fix.** Pin the image by digest, update to a current postgres image, add `cap_drop: [ALL]`, memory and CPU limits, and generate an SBOM in CI.

F-06 Wildcard CORS, no security headers, framework disclosed

Low 3.1

Category: A05 Security Misconfiguration
Vector: `CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:N/A:N`
Component: server/src/index.js

What I found. A request carrying `Origin: http://evil.example` received `Access-Control-Allow-Origin: *` (H-03). Responses include `X-Powered-By: Express` and none of Content-Security-Policy, Strict-Transport-Security, X-Content-Type-Options, X-Frame-Options or Referrer-Policy (H-04). Confidence: Certain.



H-03: wildcard CORS and X-Powered-By in the response headers

Metric	Value	Why
AV	N	Exploited through a victim's browser.
AC	H	The API has no cookies or credentials to ride, so a hostile page gains little beyond what F-01 already gives.
PR	N	No privileges needed.
UI	R	The victim must visit an attacker-controlled page.
S	U	Stays within the app.
C	L	Limited additional read exposure.
I	N	None shown.
A	N	None shown.

Fix. Restrict CORS to the known front-end origin, add Helmet, and disable `X-Powered-By`. Its main importance is future: a wildcard origin would weaken authentication once it is added.

5. What held up

Four categories were tested and found sound. These results are as important to the board as the findings, because they show where the team already did the right thing.

Test	Result	Evidence
SQL injection (A03), parameters `q`, `month`, `id`	Not vulnerable. All queries use bound parameters. The one result that looked like injection (`month=1 OR 1=1` returning all 8 rows) is explained: `Number()` gives `NaN`, the filter is skipped, and the response equals the unfiltered list. A control request (`month=1`) returned 0 rows, so the filter works. The table survived a `DROP TABLE` payload in the id.	N-01, N-01b
Stored XSS (A03)	Not vulnerable in the browser. The API stores the payload unsanitized, and React renders it as literal text. Defense in depth is thin: nothing server-side stops it if the UI ever used `dangerouslySetInnerHTML`.	N-02a, N-02b
Mass assignment and ID guessing (A01)	Not vulnerable. A supplied `id` and `isAdmin` were ignored, a random UUID was generated, and an unknown id returned 404. This does not offset F-01: the list call exposes every id anyway.	N-03
SSRF (A10)	Not applicable. No outbound request code exists in the server.	N-04
Secrets in git history	gitleaks found no leaks in the 12 commits that contain file changes (the branch has 13; one commit only adds an empty file). A rescan after the evidence commit gave the same result. The weak default password is not flagged by design and is reported in F-03.	S-02

Tested and not confirmed

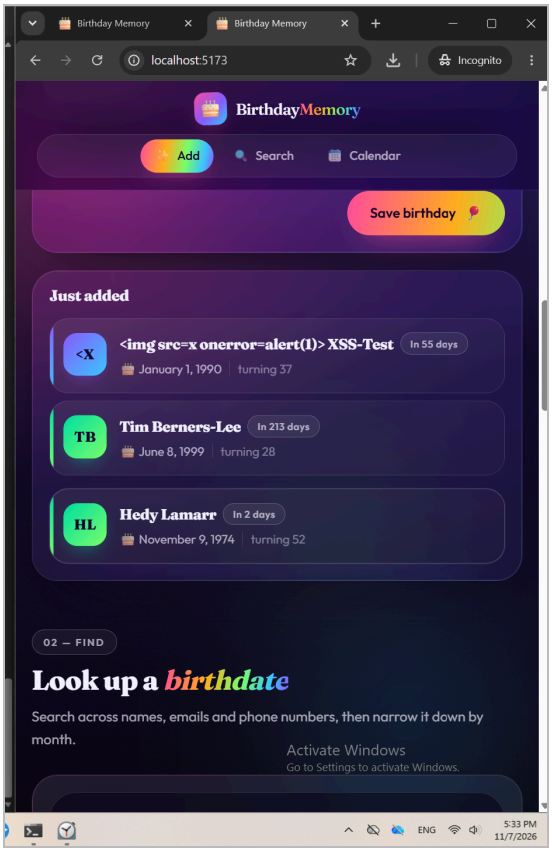
Hypothesis	Result
The health endpoint leaks database error text (H-05)	Not confirmed. With the database stopped, `/api/health` returned status 503 with an empty error string. The code does return `error.message` (index.js line 18), so another failure type could leak. An unauthenticated caller can tell when the database is down.
The database runs as root in the container (H-20)	Not confirmed. All Postgres processes run as the `postgres` user. The root user seen is only the default `docker exec` shell.
Email validation regex timing (H-07)	Not tested. Skipped as low value, to avoid an unverified claim.

Evidence for section 5

```
PS C:\Users\mcmic\source\cyber-uvu-student-projects\cybr-4550\threat_actors\birthday_memory> & { Get-Date; "--- month=1 control ---"; (curl.exe -s "http://localhost:4000/api/birthdays/?month=1" | ConvertFrom-Json).Count; "--- month=1 OR 1=1 ---"; (curl.exe -s "http://localhost:4000/api/birthdays/?month=1%20OR%201%3D1" | ConvertFrom-Json).Count; "--- no filter ---"; (curl.exe -s "http://localhost:4000/api/birthdays/" | ConvertFrom-Json).Count } | Tee-Object security\raw\N-01b.txt

Saturday, November 7, 2026 5:22:19 PM
--- month=1 control ---
0
--- month=1 OR 1=1 ---
8
--- no filter ---
8
```

N-01b: month filter control. month=1 returns 0, the payload and no filter both return 8



N-02b: the script payload displayed as literal text, no alert

```
PS C:\Users\mcmic\source\cyber-uvu-student-projects\cybr-4550\threat_actors\birthday_memory> Set-Content -Path security\raw\N-03-body.json -Encoding ascii -Value '{"id":"00000000-0000-0000-0000-000000000001","isAdmin":true,"firstName":"Extra","lastName":"Fields","birthdate":"1995-05-05","phone":"+1 415 555 0198","email":"extra@example.com"}'
PS C:\Users\mcmic\source\cyber-uvu-student-projects\cybr-4550\threat_actors\birthday_memory> & { Get-Date; curl.exe -s -i POST -H "Content-Type: application/json" --data-binary "@security\raw\N-03-body.json" http://localhost:4000/api/birthdays/; "`n--- bad id ---"; curl.exe -s -w "`nHTTP %{http_code}`n" http://localhost:4000/api/birthdays/00000000-0000-0000-0000-000000000001 } | Tee-Object security\raw\N-03.txt

Saturday, November 7, 2026 5:35:28 PM
{"id":"5346167e-0453-4fed-afab-f79508a27152","firstName":"Extra","lastName":"Fields","birthdate":"1995-05-05","phone":"+1 415 555 0198","email":"extra@example.com","createdAt":"2026-11-08T00:35:28.412Z","updatedAt":"2026-11-08T00:35:28.412Z","age":31,"turningAge":32,"daysUntil":179,"nextBirthday":"2027-05-05"}

--- bad id ---
{"error":"Not found."}
HTTP 404
```

N-03: supplied id and isAdmin ignored; unknown id returns 404

```
PS C:\Users\mcmic\source\cyber-uvu-student-projects\cybr-4550\threat_actors\birthday_memory> docker stop thebirthdates-db
thebirthdates-db
PS C:\Users\mcmic\source\cyber-uvu-student-projects\cybr-4550\threat_actors\birthday_memory> & { Get-Date; curl.exe -i http://localhost:4000/api/health } | Tee-Object security\raw\H-05.txt

Saturday, November 7, 2026 5:11:46 PM
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100    32  100    32    0     0   2575      0                              0
HTTP/1.1 503 Service Unavailable
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 32
ETag: W/"20-UchU9gUO9Ufj9k9FBVjftg7t9E"
Date: Sun, 08 Nov 2026 00:11:46 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":"degraded","error":""}
```

H-05: health endpoint with the database stopped

6. Remediation roadmap

When	Action	Closes	Effort
Today	Publish the database as `127.0.0.1:5544:5432`; stop publishing it at all if only the API needs it.	F-03 (network part)	1 line
Today	Replace the default credentials with a generated secret from the environment; remove the fallbacks in `db.js`; create a least-privilege role.	F-03	Hours
This week	Add authentication and per-record authorization to the API; return only needed columns.	F-01	Days
This week	Add request and write logging with caller identity; ship logs off the app host.	F-02	1 to 2 days
This week	Run `npm audit fix` on server and client; commit lockfiles.	F-08	Hours
This month	TLS for the API and Postgres; set `rejectUnauthorized: true`.	F-05	Days
This month	Rate limiting, pagination, body size limit; restrict CORS; add Helmet.	F-07, F-06	1 day
This month	Pin the image by digest; add capability drops, read-only filesystem and limits; generate an SBOM; run Trivy and `npm audit` in CI.	F-09, F-08	1 to 2 days
Next quarter	Encrypt the data volume and backups; consider column encryption.	F-04	Days

After the first two rows, a re-test of F-01 and F-03 should show: the database port refuses connections from the network, and every API route returns 401 without credentials.

7. Regulatory considerations

No regulation applies to this test deployment: the data is synthetic, there are no real data subjects, and nothing was exposed outside my own machine. This section is limited to what could plausibly apply if the application held real personal data.

- The application stores names, birthdates, phone numbers and email addresses. If those belonged to real people, privacy and breach-notification laws in the people's jurisdictions could apply (for example, the GDPR for people in the EU, or US state breach-notification statutes). Whether any of them applies depends on where the people live, who operates the service and thresholds that I did not assess.
- Findings F-01, F-03 and F-04 would be the relevant facts in any such assessment: unauthenticated access to personal data and plaintext storage.
- I did not assess any specific statute, and nothing here is legal advice. Counsel should be consulted before any real deployment.

8. Limitations and honest notes

- **Single machine.** All tests ran on one computer. Reachability from another device was not tested (out of scope).
- **No exploitation of dependencies, no traffic interception, no container escape.** Findings F-05, F-08 and F-09 rest on configuration, code and scanner evidence.
- **Predictions that did not hold, corrected during testing:** I expected the health endpoint to leak an error (it did not), I expected the container process to run as root (it does not), a month filter result looked like injection (it was a skipped filter), and I first said the Vite dev server listens on the network (it listens on loopback only).
- **One search came back empty without explanation** (the exact-email search of the volume, H-21). A broader search found the data.
- **Privacy of evidence.** Screenshots and two raw captures show the author's Windows username and a private LAN address (192.168.0.47). Neither is a credential. No real .env file is tracked and the real one is gitignored.
- **Scope of "full history".** The repository holds the class fork, so history is small (12 commits).
- **Not tested:** email regex timing (H-07), authentication brute force (no authentication exists), and anything requiring a second device.

Appendix A. Evidence index

Every ID has a screenshot at `security/screenshots/<ID>.png` and a raw output at `security/raw/<ID>.txt` (S-03 also has `S-03-full.txt`, the complete Trivy report). The findings register is `security/findings.md`; the coverage matrix and per-test status are in `security/EVIDENCE-LOG.md`.

ID	Shows	ID	Shows
B-01	docker compose ps, database up and healthy	H-15	No auth code
B-02	App running with seeded data	H-16a/b	No audit trail
B-03	Git baseline	H-17	No TLS
H-01	Read all records, no credentials	H-18	Certificate check disabled
H-02a	Create, no credentials (201)	H-19	Password readable by docker inspect
H-02b	Update, no credentials (200)	H-20	Container user
H-02c	Delete, no credentials (204)	H-21	Data in volume files
H-03	Wildcard CORS	H-23	Unpinned image, no SBOM
H-04	Missing headers, X-Powered-By	H-24a/b	No roles or ownership, data minimization
H-05	Health endpoint, database stopped	N-01, N-01b	SQL injection tests
H-06	No rate limit or pagination	N-02a/b	Stored XSS test
H-08a/b	Default credentials, code lines	N-03	Mass assignment, ID guessing
H-09	Superuser role	N-04	SSRF code search
H-10	Plaintext PII	S-01a/b	npm audit
H-11	LAN address answers	S-02	gitleaks
H-12	Port on all interfaces	S-03	Trivy
H-13	No container hardening	S-04, S-04b	Port check, Vite binding
H-14	Credentials in compose		

Appendix B. CVSS v3.1 vectors

ID	Score	Vector
F-01	9.8 Critical	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`
F-03	9.8 Critical	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`
F-08	7.5 High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`
F-05	6.8 Medium	`CVSS:3.1/AV:A/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N`
F-04	5.5 Medium	`CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N`
F-02	5.3 Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:N`
F-07	5.3 Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L`
F-09	3.9 Low	`CVSS:3.1/AV:L/AC:H/PR:H/UI:N/S:U/C:L/I:L/A:L`
F-06	3.1 Low	`CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:N/A:N`

> **Judgement call, F-03.** Scored AV:N because the port is published on all interfaces. If scored AV:A (same network only, since I did not test from a second device), the score is 8.8 (High). I kept AV:N

> because that is how a deployed server would be exposed, and I state the caveat in the finding.

Appendix C. AI usage disclosure

Tool. I used Claude (Anthropic) as an assistant, through the Claude desktop app and an agent environment with a shell.

What I did. I ran every test command on my own computer, took every screenshot, and sent each result back for review. I decided what to skip: I chose not to run the email-regex timing test (H-07), and it is marked NOT TESTED in the evidence log. I committed the evidence to my fork and will present and defend the work.

What the AI did. It proposed the test plan and order, read the application source, wrote the PowerShell, curl, Docker and scanner commands, kept the evidence log and tracker, drafted the findings register and the report from my test results, and I edited them. It calculated the CVSS scores with a script. It did not run any command against the target. It only saw what I sent it in screenshots and raw files.

How I checked the output.

- No finding was recorded until a request, command output or scanner report showed it. Each claim was checked against the raw output file.
- CVSS scores were calculated by a script from the vectors, not typed by hand.
- Several AI predictions were wrong and are recorded as such: leaked error text (H-05), a root process in the container (H-20), `SELECT *` in the code (the code uses a column constant), and a claim that the Vite server listens on the network (corrected by S-04b). A 6-hour versus 7-hour time offset was corrected from the screenshots.
- Scanner severity ratings are shown beside my own reachability assessment where they differ.